

Lezione 4: Esercitazione Pratica

Stream Avanzati, Collezioni, Generics e FlatMap

Michele Gentile

14 Maggio 2026

1 Analisi Performance Flotta Veicoli

1. Si crei una classe **Veicolo** con campi `targa`, `modello`, `chilometriPercorsi` (double) e `livelloCarburante` (int da 0 a 100).
2. Utilizzando uno stream, calcolare la media dei chilometri percorsi da tutti i veicoli che hanno un livello di carburante superiore al 20
3. Trovare il veicolo che ha percorso il maggior numero di chilometri in assoluto utilizzando `max` con un `Comparator`.
4. Verificare se esiste almeno un veicolo nella flotta che ha percorso più di 100.000 km utilizzando `anyMatch`.

Obiettivo: Stream Numerici e Predicati

Utilizzo di `DoubleStream` attraverso `mapToDouble` per operazioni statistiche e applicazione di predicati booleani sulla collezione.

Attenzione: Optional e Medie

Il metodo `average()` restituisce un `OptionalDouble`. Ricordate di gestire il caso in cui lo stream sia vuoto (es. nessun veicolo sopra il 20% di carburante) usando `orElse(0.0)`.

2 Sistema Universitario

1. Si definiscano due classi: **Studente** (`nome`, `matricola`) e **Corso** (`nomeCorso`, `listaIscritti` di tipo `List`).
2. Data una lista di oggetti *Corso*, utilizzare `flatMap` per ottenere uno stream unico di tutti gli studenti iscritti a tutti i corsi disponibili.
3. Dallo stream globale di studenti, rimuovere i duplicati (studenti iscritti a più corsi) basandosi sulla matricola.

4. Restituire una lista di stringhe con i nomi degli studenti, in ordine decrescente di lunghezza del nome.

Obiettivo: FlatMap e Gestione Strutture Annidate

Imparare a "appiattare" gerarchie di oggetti (una lista di liste) per eseguire analisi globali su elementi contenuti in contenitori diversi.

Tip: Il potere di FlatMap

`flatMap` trasforma ogni elemento dello stream in un altro stream e poi concatena tutti questi stream in uno solo. È fondamentale quando si passa da un oggetto "contenitore" ai suoi "contenuti".

3 Monitoraggio E-commerce: Raggruppamento

1. Si definisca una classe **Ordine** con campi `idOrdine`, `cliente`, `importo` (double) e `stato` (Enum: `SPEDITO`, `IN_ATTESA`, `ANNULLATO`).
2. Utilizzare `Collectors.groupingBy` per creare una `Map` che raggruppi gli ordini in base al loro stato.
3. Per ogni stato, calcolare la somma totale degli importi degli ordini utilizzando un "downstream collector" (`Collectors.summingDouble`).
4. Utilizzare `Collectors.partitioningBy` per dividere gli ordini in due gruppi: ordini "Premium" (importo > 500€) e ordini "Standard".

Obiettivo: Collectors Avanzati

Padronanza dei raccoglitori per trasformare stream in mappe complesse e partizionamenti logici.

Attenzione: Downstream Collectors

Il secondo argomento di `groupingBy` è opzionale: se non specificato, il valore della mappa sarà una `List`. Se volete un valore diverso (es. la somma), dovete passarlo esplicitamente.

4 Magazzino Generico con Filtri

1. Si implementi una classe generica **Magazzino** che contenga una `List` di articoli.
2. Aggiungere un metodo al magazzino che accetti un `Predicate` e restituisca una lista filtrata di elementi che soddisfano tale predicato.
3. Si testi la classe con oggetti **Prodotto** (`nome`, `scadenza`), filtrando tutti i prodotti che scadono prima di una certa data.
4. Utilizzare `findFirst()` per trovare il primo prodotto che corrisponde a un determinato criterio di ricerca, gestendo l'assenza di risultati.

⚡ Obiettivo: Generics e Interfacce Funzionali

Combinare la flessibilità dei tipi generici con la potenza delle interfacce funzionali (`Predicate`) fornite da Java.

💡 Tip: Riutilizzo del Codice

Scrivendo metodi che accettano `Predicate`, rendete il vostro magazzino compatibile con qualsiasi logica di filtro futura senza dover modificare la classe.

5 Analisi Log Server Multi-Livello

1. Si crei una classe `LogEntry` con `timestamp`, `livello` (INFO, ERROR, DEBUG), `messaggio` e `servizioSorgente`.
2. Data una lista di log, filtrare solo gli errori (ERROR).
3. Creare una stringa formattata che elenchi tutti i messaggi di errore unici, separati da " | ", limitando il risultato ai primi 5 messaggi più recenti.
4. Convertire la lista di log in una `Map<String, Long>` dove la chiave è il nome del `servizioSorgente` e il valore è il numero di log di tipo DEBUG prodotti da quel servizio.

⚡ Obiettivo: Pipeline Complesse e Limitazione

Gestione di flussi di dati reali (log) applicando filtri, ordinamenti temporali, limitazioni di volume (`limit`) e aggregazioni finali.

💡 Tip: Ordinamento Temporale

Per ordinare i log dal più recente, usate `sorted(Comparator.comparing(LogEntry::getTimestamp).reversed())`.